TERRAFORM - 05

Terraform on AWS EC2 — Complete Practical Documentation

Tasks covered:

1. Execute the script shown in the video.

2. Create an EC2 instance and install Apache HTTP Server (`httpd`) using Terraform.

3. Configure an Amazon S3 bucket as the Terraform remote backend for Task 2.

4. Configure DynamoDB state locking for Task 2.

The steps below are in simple English, with commands, file contents, explanations, and expected results. We will use Terraform to create the AWS resources instead of creating the S3 bucket and DynamoDB table manually in the AWS Console.

1. Objective

The objective of this practical is to automate AWS infrastructure using Terraform.

- Task 1: Execute the script shown in the video.
- Task 2: Create an EC2 instance and install `httpd`.
- Task 3: Store Terraform state remotely in an S3 bucket.
- Task 4: Use DynamoDB to prevent multiple Terraform operations from modifying the same state simultaneously.

2. Prerequisites

Before starting, make sure you have:

- An AWS account.
- AWS CLI installed and configured.
- Terraform installed on your computer.
- An AWS IAM user or role with the required permissions to create EC2, S3, DynamoDB, and related resources.
- An internet connection.

Verify the installations in Windows PowerShell:

terraform -version
aws --version
aws sts get-caller-identity

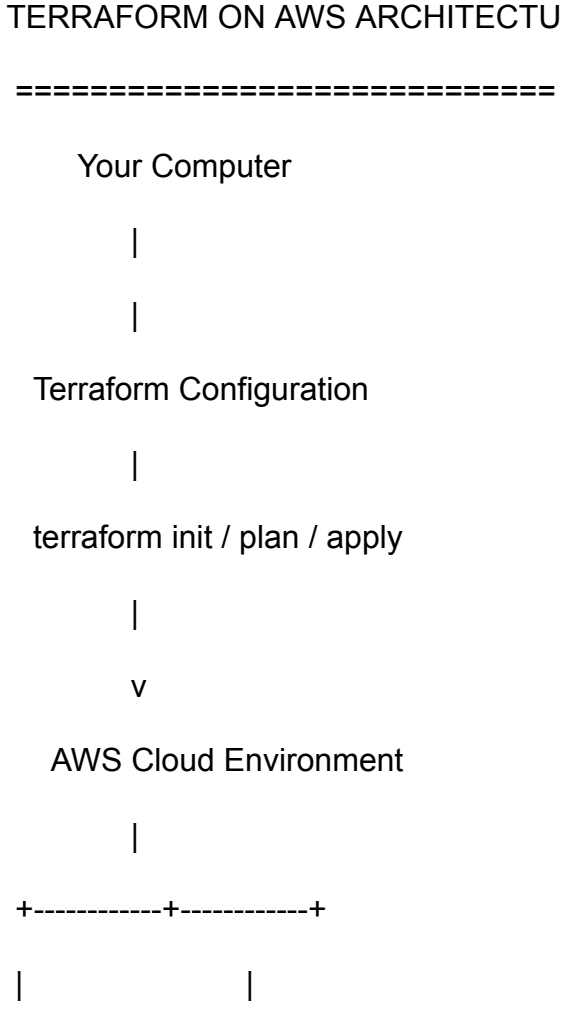
Expected result: Terraform and AWS CLI versions should appear, and the last command should display your AWS account and identity.

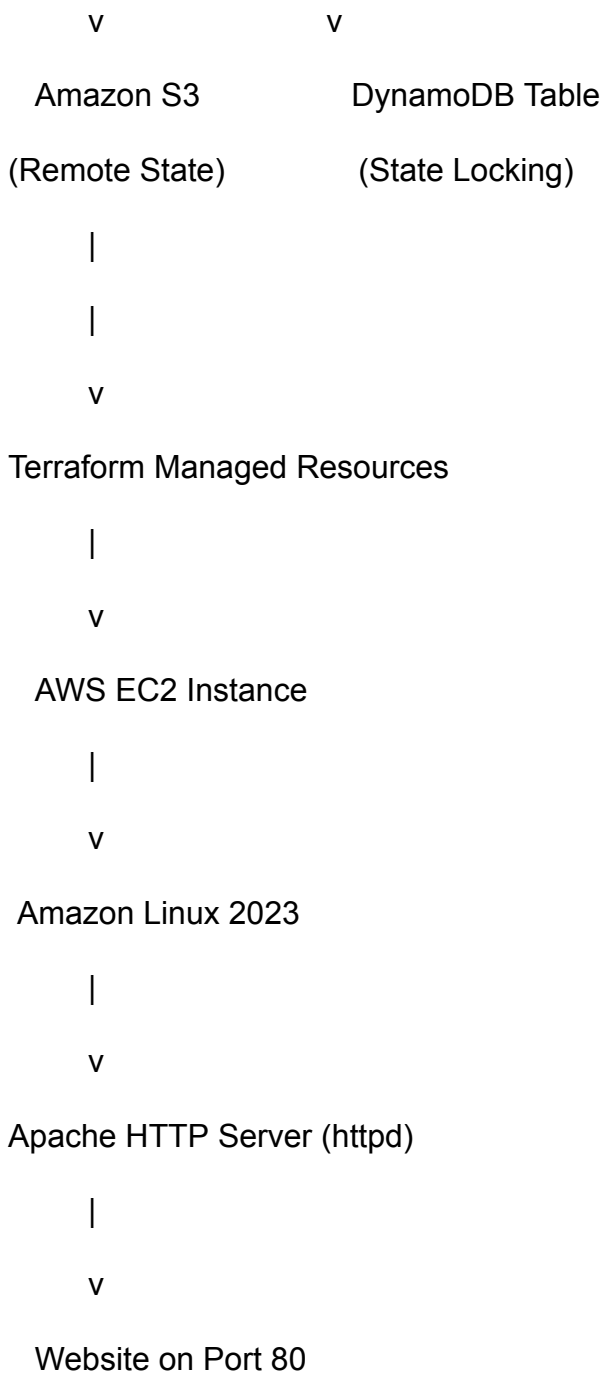
If AWS CLI is not configured, run:

aws configure

Enter your AWS access key, secret key, default region (for example, `us-east-1`), and output format. Avoid sharing credentials or saving them in Terraform files.

3. Understand the architecture





Important: S3 stores the state file. DynamoDB locking is supported by the older S3 backend locking configuration, but it is deprecated in newer Terraform versions. Current Terraform versions support native S3 lockfiles using `use_lockfile = true`. We will use the requested DynamoDB method and explain the modern alternative at the end.

4. Task 1 — Execute the script shown in the video

Creating S3 , Dynabo db using terraform and S3 as backend for terraform.tfstate file

Creating S3

```
resource "aws_s3_bucket" "s3_bucket" {
  bucket = "s3backend"
}
```

After that terraform init

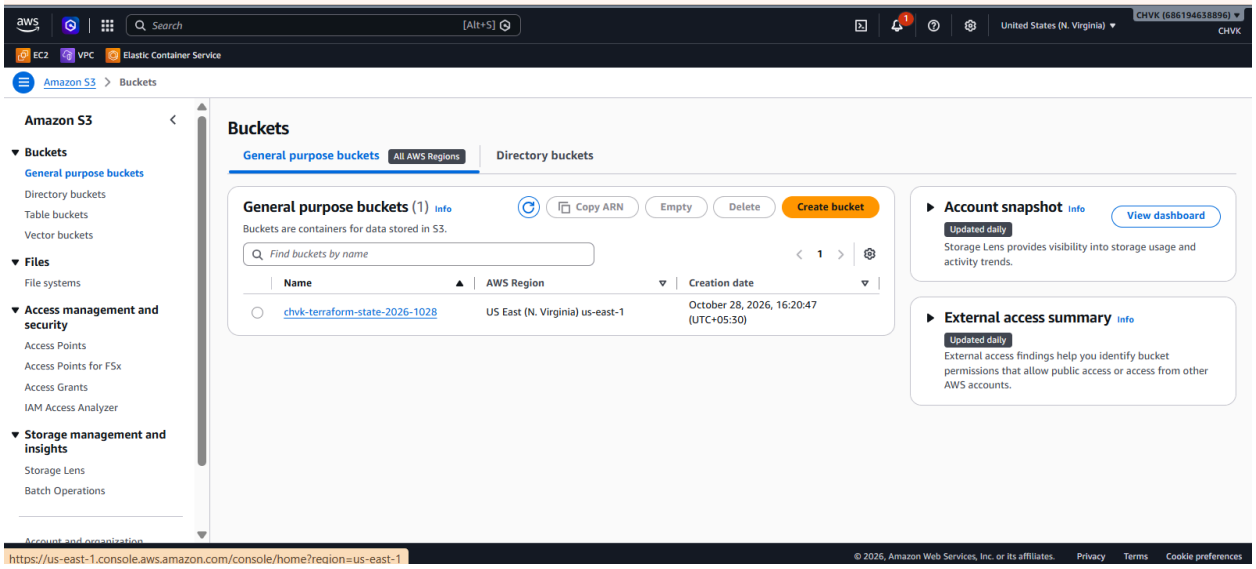
terraform apply

It will create a bucket on yours aws account

To see that login to aws console

Search s3

On s3 you can see general purpose bucket

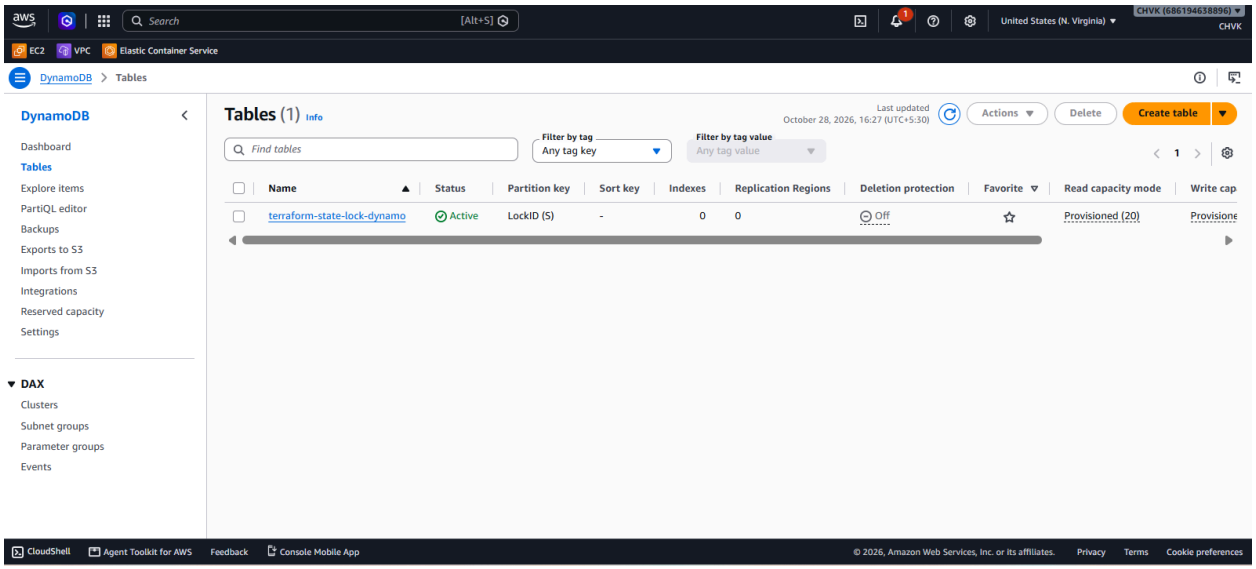


Creating Dynamo db

```
resource "aws_s3_bucket" "s3_bucket" {
  bucket = "chvk-terraform-state-2026-1028"
}
resource "aws_dynamodb_table" "dynamodb_terraform_state_lock"{
  name = "terraform-state-lock-dynamo"
  hash_key = "LockID"
  read_capacity = 20
  write_capacity = 20
  attribute {
    name = "LockID"
    type = "S"
  }
```

```
}
```

After that terraform apply
It will create a bucket on yours aws account
To see that login to aws console
Search dynamodb
You can see tables



S3 backend for for terraform.tfstate file

```
resource "aws_s3_bucket" "s3_bucket" {
  bucket = "chvk-terraform-state-2026-1028"
}
resource "aws_dynamodb_table" "dynamodb_terraform_state_lock"{
  name = "terraform-state-lock-dynamo"
  hash_key = "LockID"
  read_capacity = 20
  write_capacity = 20
  attribute {
    name = "LockID"
    type = "S"
  }
}
terraform {
  backend "s3" {
    bucket         = "chvk-terraform-state-2026-1028"
    key            = "terraform.tfstate"
    region         = "us-east-1"
```

```
    dynamodb_table = "terraform-state-lock-dynamo"
  }
}
```

After that terraform init

terraform apply

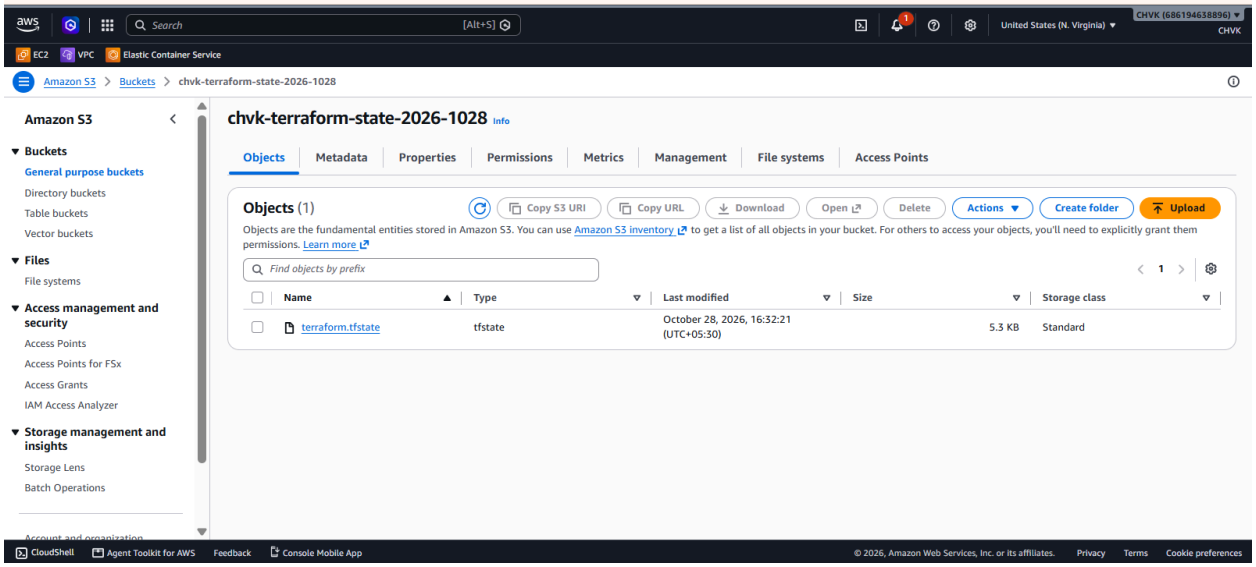
It will create a bucket on yours aws account

To see that login to aws console

Search s3

On s3 you can see general purpose bucket

Open your bucket on that you can see your terraform.tfsate file



Terraform provisioners

We have two type

1) Local provisioner

This is used to execute commands at the run time on local machine. (means where terraform is installed)

Example of Local provisioner:

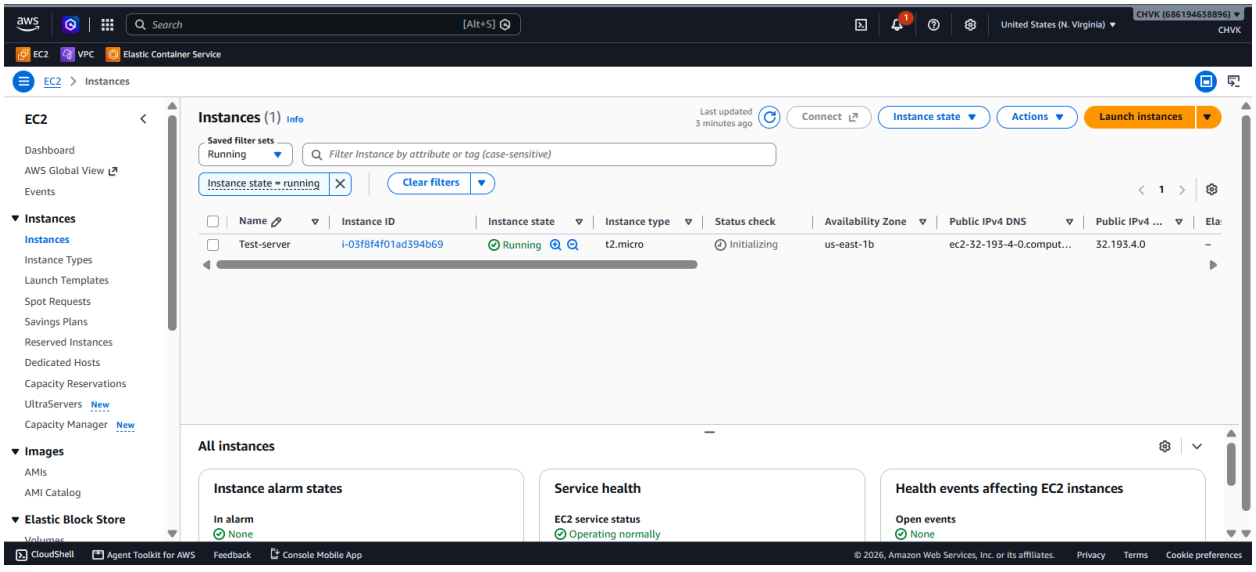
==========

```
resource "aws_instance" "test-server" {
 ami = "ami-005f9685cb30f234b"
 instance_type = "t2.micro"
 key_name = "linux-01"
 tags = {
  Name = "Test-server"
    }
   provisioner "local-exec" {
     command = "echo Instance ${aws_instance.test-server.public_ip} created! >
instance_state.txt"
```

```
}
}
```

```
main.tf
main.tf > resource "aws_instance" "test-server" > key_name
1   resource "aws_instance" "test-server" {
2     ami = "ami-0d97d7f07b25dd693"
3     instance_type = "t2.micro"
4     key_name = "jenkins-key04"
5     tags = {
6      Name = "Test-server"
7       }
8       provisioner "local-exec" {
9         command = "echo Instance ${aws_instance.test-server.public_ip} created! > instance_
10
11    }
12  }
13
```

In my case i will use existing key pair if we want create ,create the new one and add that
After that use these commands
terraform init
terraform plan
terraform apply
It will automatically creates the new instance
You want to check that got to aws console and open ec2 on that you can see running insdtance with name Test-server



2) Remote provisioner

This is used to execute commands at the run time on remote machines.

Example of remote provisioner:

==========

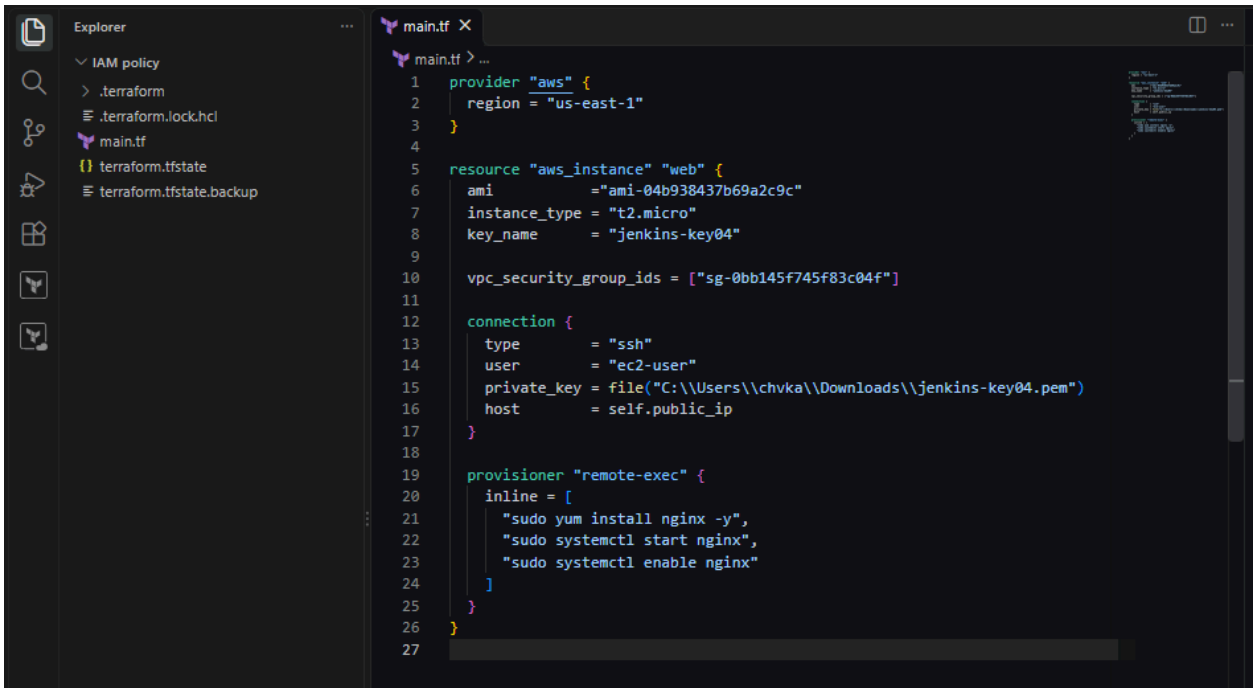
```
provider "aws" {
  region = "ap-south-1"
}

resource "aws_instance" "web" {
  ami           = "ami-xxxxxxxx"
  instance_type = "t2.micro"
  key_name      = "my-key"

  vpc_security_group_ids = ["sg-xxxxxxxx"]

  connection {
    type        = "ssh"
    user        = "ec2-user"
    private_key = file("my-key.pem")
    host        = self.public_ip
  }

  provisioner "remote-exec" {
    inline = [
      "sudo yum install nginx -y",
      "sudo systemctl start nginx",
      "sudo systemctl enable nginx"
    ]
  }
}
```



In my case i will use existing key pair and existing sg id if we want create ,create the new one and add that

After that use these commands

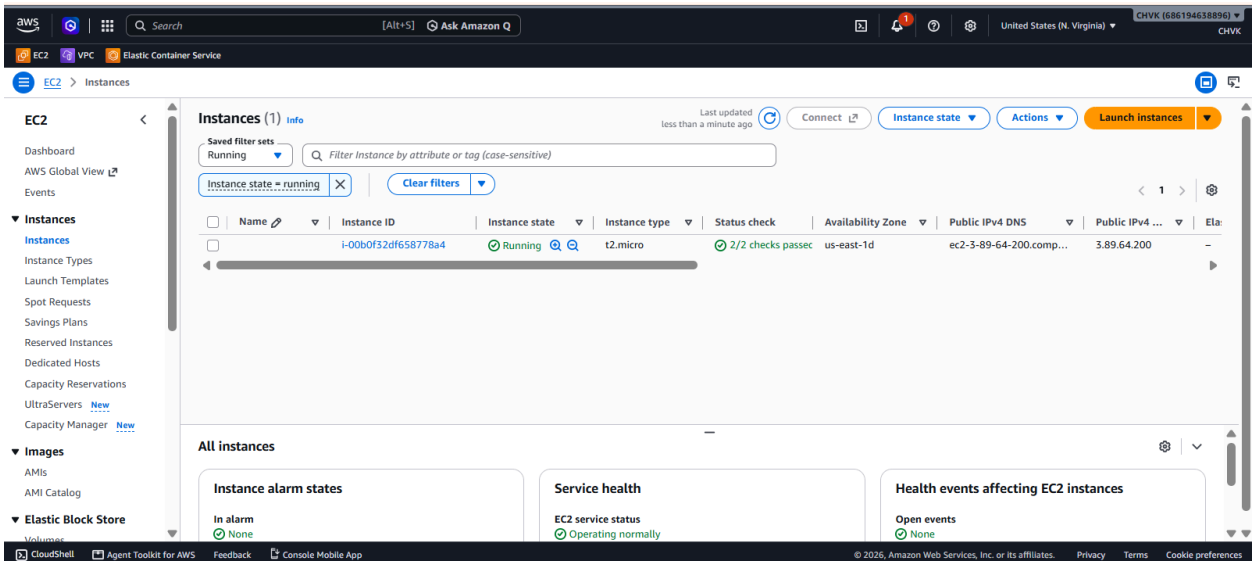
terraform init

terraform plan

terraform apply

It will create one ec2 instance and starts the enginx

If we want to see thet instance goto aws console go to ec2 instance you can see new indstance



Next open thet instance copt <public_ip>

Open browser and search http://<ec2 public_ip>:80

It will open a nignx page



What is the Difference Between Taint and Untaint?

Command
Action Performed
What happens on next terraform apply?
terraform taint
Manually marks a specific resource as degraded, broken, or out-of-sync in the local state file.
Terraform treats the resource as destroyed and forces it to be deleted and recreated from scratch.
terraform untaint
Reverses the tainted status, returning the resource back to its normal, healthy state.
Terraform cancels the scheduled destruction/recreation; the resource is left completely untouched.

When and Where to Use It?

• When a resource configuration fails halfway: If a provisioning script or configuration crashes during creation, Terraform automatically marks that resource as tainted so it can clean up the broken state on the next run..

• When manual changes are made outside of Terraform: If an engineer manually logs into AWS and modifies an EC2 server (e.g., deletes core packages, messes up configurations, or breaks dependencies), Terraform won't automatically notice if the code looks the same. By running terraform taint, you force Terraform to throw away the broken instance and spin up a fresh, clean replacement..

Step-by-Step Code and Execution Guide

Step 1: Create your project directory and configuration file

Open your Mac Terminal and run:
Bash
mkdir -p terraform-taint-demo
cd terraform-taint-demo

Create a main.tf file using your text editor and add the following code:
Terraform
provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "webserver" {
  ami   = "ami-02f3f602d23f1659d" # Amazon Linux 2023 AMI (us-east-1)
  instance_type = "t2.micro"

  tags = {
    Name = "Taint-Demo-Server"
  }
}

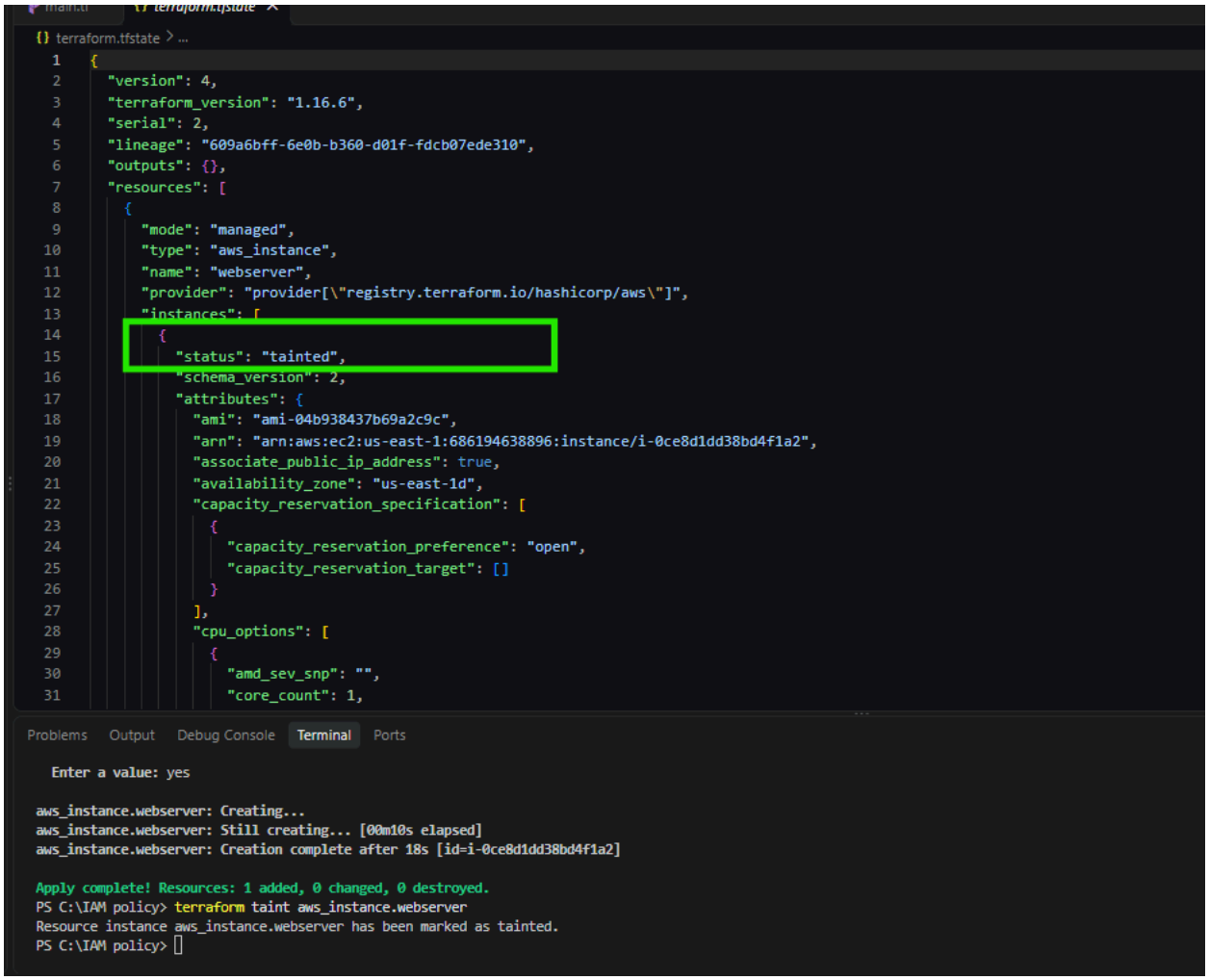


Step 2: Initialize and apply the configuration
Initialize your working directory and provision the EC2 instance:
Bash
terraform init
terraform apply

Type yes when prompted. Note down or verify that the EC2 instance is successfully created.

Step 3: Use terraform taint
Imagine something went wrong with this server manually, or you want to force Terraform to rebuild it on the next run. Run the taint command referencing the resource type and name (aws_instance.webserver):

Bash
terraform taint aws_instance.webserver



What you will see in the terminal:
Resource instance aws_instance.webserver has been marked as tainted.
If you run terraform plan right now, Terraform will explicitly state that it plans to destroy and recreate (-/+) the instance.

Step 4: Use terraform untaint (Optional Reversal)

If you change your mind and decide you do not want to destroy and recreate the resource after all, you can undo the taint status:
Bash
terraform untaint aws_instance.webserver

```
PS C:\IAM policy> terraform untaint aws_instance.webserver
Resource instance aws_instance.webserver has been successfully untainted.
PS C:\IAM policy>
```

What you will see in the terminal:
Resource instance aws_instance.webserver has been successfully untainted.
If you run terraform plan now, the plan will change back to "No changes. Infrastructure is up-to-date."

Debugging:
==========
terraform apply will provide us the logs/cause of the issue,
but still if we want to dig deeper then we need to export a varibale

Set-Item -Path env:TF_LOG -value "TRACE"
It will give full details

```
3+0530"
2026-10-28T17:51:32.925+0530 [TRACE] provider.terraform-provider-aws_v6.70.0_x5.exe: Transforming empty block to null block: @caller=github.com/hashicorp/terraform-plugin-framework@v1.19.0/internal/fwschemadata/data_nullify_collection_blocks.go:91 @module=sdk.framework description=configuration tf_attribute_path=ignore_tags tf_req_id=8655201b-e856-c8cb-71ce-ee81d974e189 tf_rpc=ConfigureProvider tf_mux_provider="*proto5server.Server" tf_provider_addr=registry.terraform.io/hashicorp/aws timestamp="2026-10-28T17:51:32.924+0530"
2026-10-28T17:51:32.925+0530 [TRACE] provider.terraform-provider-aws_v6.70.0_x5.exe: Transforming empty block to null block: tf_req_id=8655201b-e856-c8cb-71ce-ee81d974e189 @module=sdk.framework tf_provider_addr=registry.terraform.io/hashicorp/aws tf_rpc=ConfigureProvider @caller=github.com/hashicorp/terraform-plugin-framework@v1.19.0/internal/fwschemadata/data_nullify_collection_blocks.go:91 description=configuration tf_attribute_path=assume_role tf_mux_provider="*proto5server.Server" timestamp="2026-10-28T17:51:32.925+0530"
2026-10-28T17:51:32.925+0530 [TRACE] provider.terraform-provider-aws_v6.70.0_x5.exe: Transforming empty block to null block: tf_provider_addr=registry.terraform.io/hashicorp/aws tf_rpc=ConfigureProvider description=configuration tf_attribute_path=assume_role_with_web_identity tf_req_id=8655201b-e856-c8cb-71ce-ee81d974e189 @caller=github.com/hashicorp/terraform-plugin-framework@v1.19.0/internal/fwschemadata/data_nullify_collection_blocks.go:91 @module=sdk.framework tf_mux_provider="*proto5server.Server" timestamp="2026-10-28T17:51:32.925+0530"
2026-10-28T17:51:32.925+0530 [TRACE] provider.terraform-provider-aws_v6.70.0_x5.exe: Transforming empty block to null block: description=configuration tf_attribute_path=default_tags tf_mux_provider="*proto5server.Server" tf_provider_addr=registry.terraform.io/hashicorp/aws tf_req_id=8655201b-e856-c8cb-71ce-ee81d974e189 tf_rpc=ConfigureProvider @caller=github.com/hashicorp/terraform-plugin-framework@v1.19.0/internal/fwschemadata/data_nullify_collection_blocks.go:91 @module=sdk.framework timestamp="2026-10-28T17:51:32.925+0530"
2026-10-28T17:51:32.926+0530 [TRACE] provider.terraform-provider-aws_v6.70.0_x5.exe: Transforming empty block to null block: @module=sdk.framework description=configuration tf_mux_provider="*proto5server.Server" tf_req_id=8655201b-e856-c8cb-71ce-ee81d974e189 tf_attribute_path=endpoints tf_provider_addr=registry.terraform.io/hashicorp/aws tf_rpc=ConfigureProvider @caller=github.com/hashicorp/terraform-plugin-framework@v1.19.0/internal/fwschemadata/data_nullify_collection_blocks.go:91 timestamp="2026-10-28T17:51:32.925+0530"
2026-10-28T17:51:32.926+0530 [TRACE] provider.terraform-provider-aws_v6.70.0_x5.exe: Calling provider defined Provider Configure: tf_provider_addr=registry.terraform.io/hashicorp/aws tf_req_id=8655201b-e856-c8cb-71ce-ee81d974e189 @caller=github.com/hashicorp/terraform-plugin-framework@v1.19.0/internal/fwserver/server_configureprovider.go:15 tf_mux_provider="*proto5server.Server" tf_rpc=ConfigureProvider @module=sdk.framework timestamp="2026-10-28T17:51:32.925+0530"
2026-10-28T17:51:32.926+0530 [TRACE] provider.terraform-provider-aws_v6.70.0_x5.exe: Called provider defined Provider Configure: tf_mux_provider="*proto5server.Server" tf_req_id=8655201b-e856-c8cb-71ce-ee81d974e189 @caller=github.com/hashicorp/terraform-plugin-framework@v1.19.0/internal/fwserver/server_configureprovider.go:23 tf_provider_addr=registry.terraform.io/hashicorp/aws tf_rpc=ConfigureProvider @module=sdk.framework timestamp="2026-10-28T17:51:32.926+0530"
2026-10-28T17:51:32.926+0530 [TRACE] provider.terraform-provider-aws_v6.70.0_x5.exe: Received downstream response: tf_rpc=Configure @module=sdk.proto diagnostic_warning_count=0 tf_provider_addr=registry.terraform.io/hashicorp/aws @caller=github.com/hashicorp/terraform-plugin-go@v0.31.0/tfprotov5/internal/tf5serverlogging/downstream_request.go:61 diagnostic_error_count=0 tf_proto_version=5.11 tf_req_duration_ms=1235 tf_req_id=8655201b-e856-c8cb-71ce-ee81d974e189 timestamp="2026-10-28T17:51:32.926+0530"
2026-10-28T17:51:32.926+0530 [DEBUG] No policy client configured, skipping policy evaluation for provider["registry.terraform.io/hashicorp/aws"]
2026-10-28T17:51:32.926+0530 [TRACE] provider.terraform-provider-aws_v6.70.0_x5.exe: Served request: @module=sdk.proto tf_req_id=8655201b-e856-c8cb-71ce-ee81d974e189 tf_rpc=Configure tf_proto_version=5.11 tf_provider_addr=registry.terraform.io/hashicorp/aws @caller=github.com/hashicorp/terraform-plugin-go@v0.31.0/tfprotov5/tf5server/server.go:646 timestamp="2026-10-28T17:51:32.926+0530"
2026-10-28T17:51:32.926+0530 [TRACE] vertex "provider[\"registry.terraform.io/hashicorp/aws\"]": visit complete
2026-10-28T17:51:32.927+0530 [TRACE] vertex "aws_instance.webserver (expand)": starting visit (*terraform.nodeExpandApplyableResource)
2026-10-28T17:51:32.927+0530 [TRACE] vertex "aws_instance.webserver (expand)": does not belong to any module instance
2026-10-28T17:51:32.928+0530 [TRACE] vertex "aws_instance.webserver (expand)": expanding dynamic subgraph
2026-10-28T17:51:32.928+0530 [TRACE] vertex "aws_instance.webserver (expand)": produced no dynamic subgraph
2026-10-28T17:51:32.928+0530 [TRACE] vertex "aws_instance.webserver (expand)": visit complete
2026-10-28T17:51:32.928+0530 [TRACE] vertex "provider[\"registry.terraform.io/hashicorp/aws\"] (close)": starting visit (*terraform.graphNodeCloseProvider)
2026-10-28T17:51:32.928+0530 [TRACE] vertex "provider[\"registry.terraform.io/hashicorp/aws\"] (close)": does not belong to any module instance
2026-10-28T17:51:32.928+0530 [TRACE] GRPCProvider: Close
2026-10-28T17:51:32.930+0530 [DEBUG] provider.stdio: received EOF, stopping recv loop: err="rpc error: code = Unavailable desc = error reading from server: EOF"
2026-10-28T17:51:33.021+0530 [INFO]  provider: plugin process exited: plugin=.terraform/providers/registry.terraform.io/hashicorp/aws/6.70.0/windows_amd64/terraform-provider-aws_v6.70.0_x5.exe id=29316
2026-10-28T17:51:33.021+0530 [DEBUG] provider: plugin exited
2026-10-28T17:51:33.021+0530 [TRACE] vertex "provider[\"registry.terraform.io/hashicorp/aws\"] (close)": visit complete
2026-10-28T17:51:33.022+0530 [TRACE] vertex "root": starting visit (*terraform.nodeCloseModule)
2026-10-28T17:51:33.022+0530 [TRACE] vertex "root": does not belong to any module instance
2026-10-28T17:51:33.022+0530 [TRACE] vertex "root": visit complete
2026-10-28T17:51:33.024+0530 [TRACE] statemgr.Filesystem: not making a backup, because the new snapshot is identical to the old
2026-10-28T17:51:33.025+0530 [TRACE] statemgr.Filesystem: no state changes since last snapshot
2026-10-28T17:51:33.026+0530 [TRACE] statemgr.Filesystem: writing snapshot at terraform.tfstate
2026-10-28T17:51:33.029+0530 [TRACE] statemgr.Filesystem: removing lock metadata file .terraform.tfstate.lock.info
```

To store the logs permanently then we can export a path
Set-Item -Path env:TF_LOG_PATH -value "terraform.log"

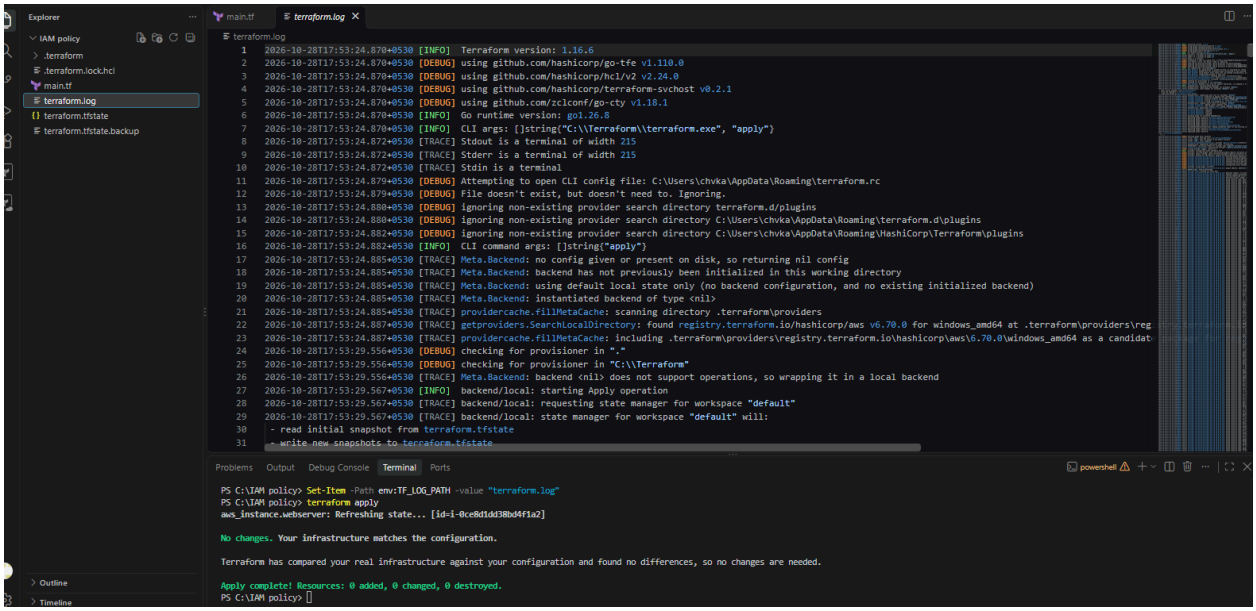
It will a one file on that it will store all details

```
PS C:\IAM policy> Set-Item -Path env:TF_LOG_PATH -value "terraform.log"
PS C:\IAM policy> terraform apply
aws_instance.webserver: Refreshing state... [id=i-0ce8d1dd38bd4f1a2]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
PS C:\IAM policy>
```



Terraform Import — Step-by-Step Guide

1. What is Terraform Import?

Terraform Import is used to bring an existing AWS resource under Terraform management without creating a new resource.

Example: You already have an EC2 instance in AWS, and you want to manage it using Terraform. You can import that instance into Terraform.

2. Steps to Import an EC2 Instance

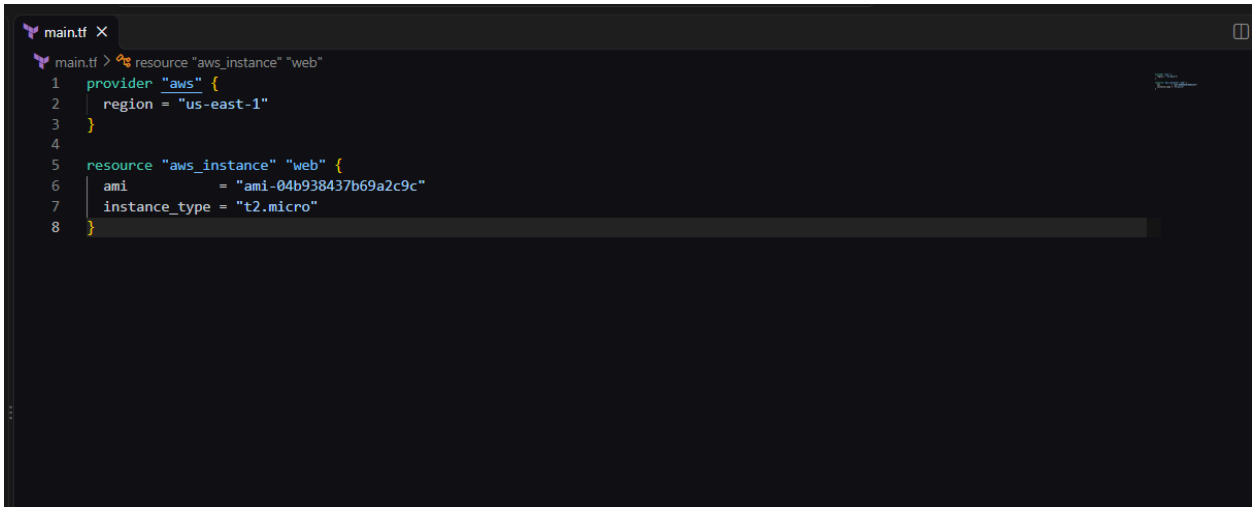
Step 1: Create a Terraform configuration

Create a `main.tf` file in your working folder.

```
provider "aws" {
  region = "us-east-1"
```

```
}

resource "aws_instance" "web" {
  ami           = "ami-04b938437b69a2c9c"
  instance_type = "t2.micro"
}
```



The resource block tells Terraform which resource you want to manage. Update the AMI and other settings to match your existing EC2 instance.

Step 2: Initialize Terraform

Run this command in PowerShell:

terraform init

This initializes Terraform and downloads the required provider.

Step 3: Import the existing EC2 instance

First, find your EC2 instance ID in AWS Console → EC2 → Instances.

Then run:

terraform import aws_instance.web i-0123456789abcdef0

Replace `i-0123456789abcdef0` with your actual EC2 instance ID.

- `aws_instance.web` — the resource address in your Terraform configuration.
- `i-0123456789abcdef0` — the existing EC2 instance ID.

```
PS C:\import> terraform import aws_instance.web i-0ce8d1dd38bd4f1a2
aws_instance.web: Importing from ID "i-0ce8d1dd38bd4f1a2"...
aws_instance.web: Import prepared!
  Prepared aws_instance for import
aws_instance.web: Refreshing state... [id=i-0ce8d1dd38bd4f1a2]

Import successful!

The resources that were imported are shown above. These resources are now in
your Terraform state and will henceforth be managed by Terraform.
```

Step 4: Check the Terraform state

terraform state list

Expected output:

Aws_instance.web

```
PS C:\import> terraform state list
aws_instance.web
```

This confirms that Terraform now tracks the imported instance.

Step 5: Check the configuration

terraform plan

Review the plan carefully. If Terraform proposes changes, update `main.tf` to match the existing resource before applying anything.

```
PS C:\import> terraform plan
aws_instance.web: Refreshing state... [id=i-0ce8d1dd38bd4f1a2]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  ~ update in-place

Terraform will perform the following actions:

  # aws_instance.web will be updated in-place
  ~ resource "aws_instance" "web" {
        id                                   = "i-0ce8d1dd38bd4f1a2"
      ~ tags                                 = {
          - "Name" = "Taint-Demo-Server" -> null
        }
      ~ tags_all                             = {
          - "Name" = "Taint-Demo-Server" -> null
        }
      + user_data_replace_on_change          = false
        # (38 unchanged attributes hidden)

        # (9 unchanged blocks hidden)
    }

Plan: 0 to add, 1 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
PS C:\import>
```

3. Important Points

- Terraform import does not create a new EC2 instance.
- Import adds the existing resource to Terraform state.
- You must define the corresponding resource block in `main.tf`.
- The configuration should match the actual AWS resource to avoid unwanted changes.
- Do not run `terraform apply` until you understand the proposed changes.

In simple words: Terraform Import connects an existing AWS resource to Terraform so you can manage it using Terraform commands.

5. Task 2 — Create an EC2 instance with httpd using Terraform

Step 1: Create a project folder

Open Windows PowerShell and run:

```
mkdir C:\IAM policy
cd C:\IAM policy
code .
```

This opens the folder in VS Code. If `code .` does not work, open VS Code manually and select this folder.

Create a file named `main.tf`.

Step 2: Write the Terraform script

Copy the following code into `main.tf`.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
```

```
}

provider "aws" {
  region = "us-east-1"
}

# Find the latest Amazon Linux 2023 AMI
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
  }

  filter {
    name   = "architecture"
    values = ["x86_64"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}

# Create a Security Group
resource "aws_security_group" "web_sg" {
  name        = "terraform-httpd-sg"
  description = "Allow HTTP traffic"

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
```

```
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

# Create EC2 and install Apache HTTP Server
resource "aws_instance" "web_server" {
  ami                    = data.aws_ami.amazon_linux.id
  instance_type          = "t2.micro"
  vpc_security_group_ids = [aws_security_group.web_sg.id]

  user_data = <<-EOF
    #!/bin/bash
    dnf install -y httpd
    systemctl enable --now httpd
    echo "Hello! Apache installed using Terraform." > /var/www/html/index.html
  EOF

  tags = {
    Name = "Terraform-HTTPD-Server"
  }
}

# Display the EC2 public IP
output "instance_public_ip" {
  value = aws_instance.web_server.public_ip
}

# Display the website URL
output "website_url" {
  value = "http://${aws_instance.web_server.public_dns}"
}
```

Step 3: Understand the important parts

Code	Purpose

`provider "aws"`	Connects Terraform to AWS in the selected region.
`data "aws_ami"`	Finds a suitable Amazon Linux 2023 image.
`aws_security_group`	Allows incoming HTTP traffic on port 80.
`aws_instance`	Creates the EC2 instance.
`user_data`	Installs and starts Apache automatically when the instance launches.
`output`	Displays the public IP address and website URL.

Why use `user_data`? It lets EC2 run the installation script during initial startup, so you do not need to connect manually and install Apache.

Step 4: Initialize Terraform

Run these commands in PowerShell, inside `C:\terraform-ec2`:

terraform init

Purpose: Downloads the AWS provider and initializes the Terraform working directory.

Expected result: Terraform initializes successfully.

Step 5: Validate the script

terraform validate

Purpose: Checks whether the Terraform configuration is syntactically valid.

Expected result:

Success! The configuration is valid.

Step 6: Review the execution plan

terraform plan

Purpose: Shows what Terraform intends to create before making changes.

Check that the plan includes an EC2 instance and security group.

Step 7: Create the EC2 instance

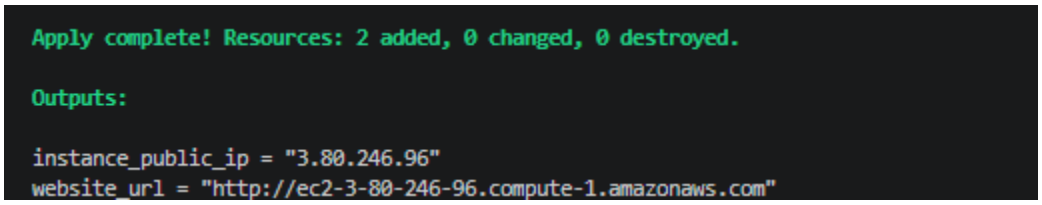
terraform apply

Terraform displays a plan and asks for confirmation. Type:

yes

Terraform will create the security group and EC2 instance. The instance may need a few minutes to finish booting and installing Apache.

Expected result: the apply operation completes successfully and prints the public IP and website URL.

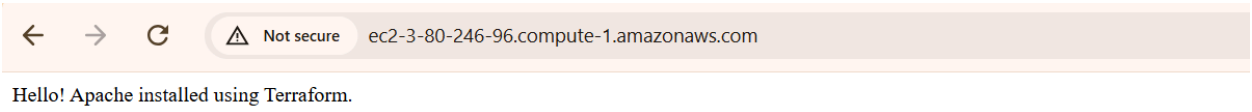


Step 8: Verify Apache in the browser

Open the `website_url` printed by Terraform in your browser.

Expected page:

Hello! Apache installed using Terraform.



This confirms that the EC2 instance is running Apache and serving the test page.

If the page does not load immediately, wait a couple of minutes and try again. Confirm that the EC2 instance has a public IP, internet access, and inbound TCP port 80 allowed.

Task 2 is complete. Terraform created the EC2 instance and installed Apache automatically.

6. Task 3 — Configure S3 as the Terraform remote backend

What is a backend?

Terraform normally stores its state in a local file named `terraform.tfstate`. A backend determines where that state is stored.

An S3 backend stores the state file in an Amazon S3 bucket instead of keeping it only on your computer. This helps teams share the state securely.

Important: Why do we need two folders?

Terraform cannot use an S3 bucket as its backend before that bucket exists. To avoid creating the bucket manually, we will create the S3 bucket and DynamoDB table using a separate Terraform configuration first.

We will use this structure:

```
C:\
├── terraform-ec2\
│   └── main.tf
│
```

└── terraform-bootstrap\
    └── main.tf

The bootstrap folder creates the S3 bucket and DynamoDB table. The original EC2 project will then use those resources for remote state and locking.

Step 1: Create the bootstrap folder

Open PowerShell:

```
mkdir C:\terraform-bootstrap
cd C:\terraform-bootstrap
code .
```

Create a file named `main.tf` in this folder.

Step 2: Create the S3 bucket using Terraform

Paste this code into `C:\terraform-bootstrap\main.tf`.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

# Create S3 bucket for Terraform state
resource "aws_s3_bucket" "terraform_state" {
  bucket = "chvk-terraform-state-CHANGE-12345"

  lifecycle {
    prevent_destroy = true
  }
```

```
  tags = {
    Name = "Terraform Remote State"
  }
}

# Enable bucket versioning
resource "aws_s3_bucket_versioning" "state_versioning" {
  bucket = aws_s3_bucket.terraform_state.id

  versioning_configuration {
    status = "Enabled"
  }
}

# Enable server-side encryption
resource "aws_s3_bucket_server_side_encryption_configuration" "state_encryption" {
  bucket = aws_s3_bucket.terraform_state.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

# Block public access
resource "aws_s3_bucket_public_access_block" "state_public_access" {
  bucket = aws_s3_bucket.terraform_state.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

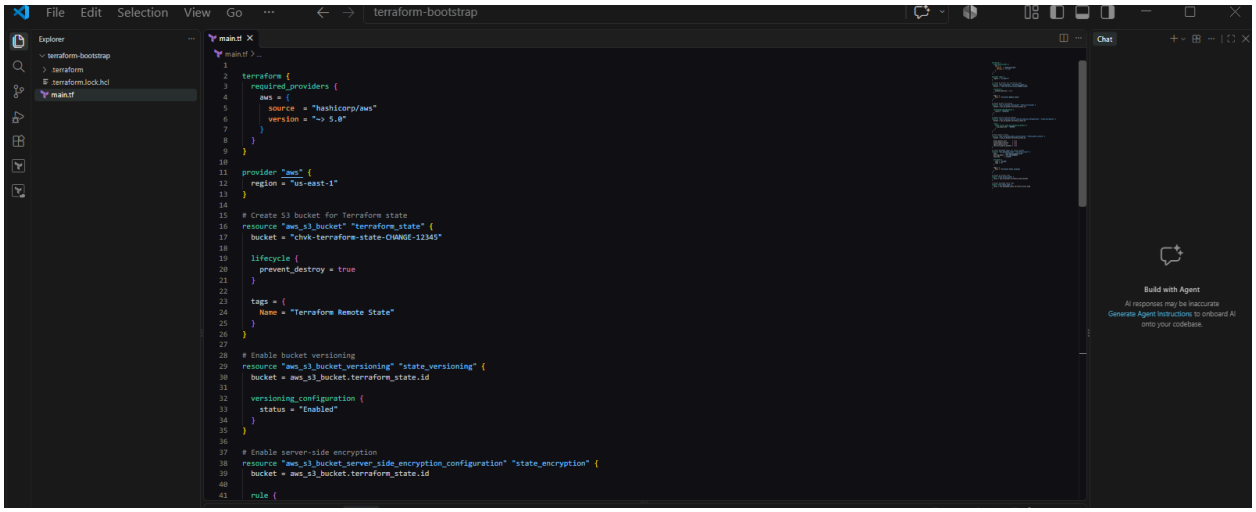
# Create DynamoDB table for state locking
resource "aws_dynamodb_table" "terraform_locks" {
  name         = "terraform-state-locks"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "LockID"
```

```
  attribute {
    name = "LockID"
    type = "S"
  }

  tags = {
    Name = "Terraform State Locking"
  }
}

# Output S3 bucket name
output "state_bucket_name" {
  value = aws_s3_bucket.terraform_state.bucket
}

# Output DynamoDB table name
output "lock_table_name" {
  value = aws_dynamodb_table.terraform_locks.name
}
```



Before applying: Change `chvk-terraform-state-CHANGE-12345` to a globally unique bucket name. S3 bucket names must be unique across AWS accounts, so if this name is already taken, choose another. Keep the chosen name for the next steps.

Step 3: Initialize the bootstrap project

cd C:\terraform-bootstrap

terraform init

Step 4: Validate the configuration

terraform validate

Expected result:

Success! The configuration is valid.

Step 5: Preview the resources

terraform plan

Check that the plan includes:

- One S3 bucket.
- S3 bucket versioning.
- S3 default encryption.
- S3 public access blocking.
- One DynamoDB table.

Step 6: Create the resources automatically

terraform apply

Review the plan, type `yes`, and wait for completion.

Terraform creates both the S3 bucket and DynamoDB table. You do not need to create either resource manually in the AWS Console.

Step 7: Record the output

Terraform should display output similar to:

state_bucket_name = "your-unique-bucket-name"
lock_table_name   = "terraform-state-locks"

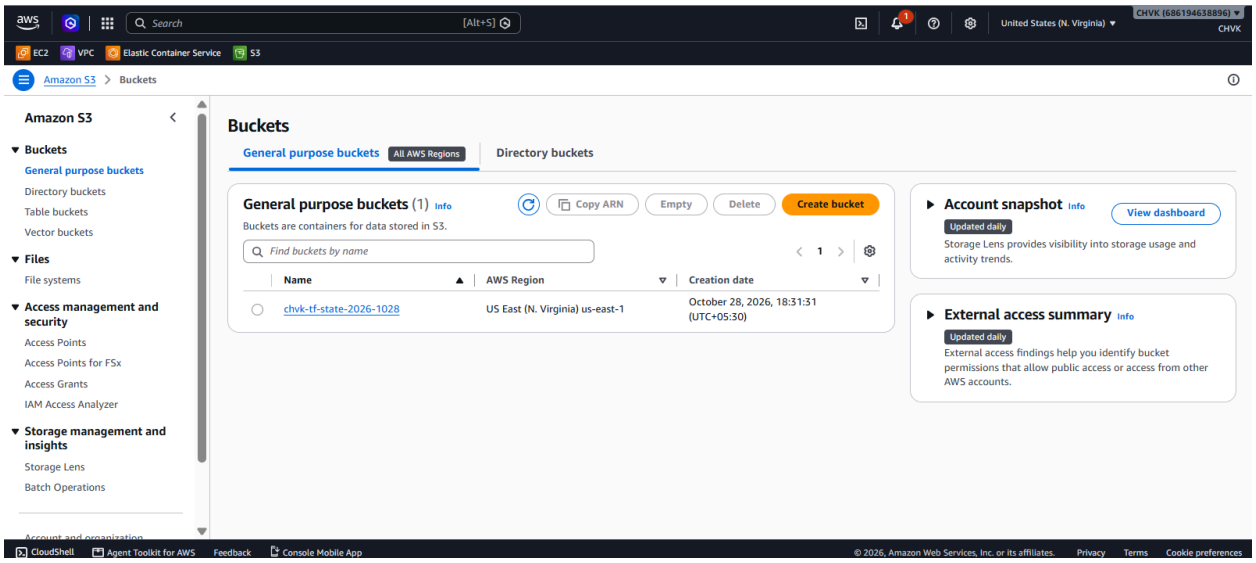
```
aws_s3_bucket.terraform_state: Creating...
aws_s3_bucket.terraform_state: Creation complete after 8s [id=chvk-tf-state-2026-1028]
aws_s3_bucket_public_access_block.state_public_access: Creating...
aws_s3_bucket_server_side_encryption_configuration.state_encryption: Creating...
aws_s3_bucket_versioning.state_versioning: Creating...
aws_s3_bucket_server_side_encryption_configuration.state_encryption: Creation complete after 2s [id=chvk-tf-state-2026-1028]
aws_s3_bucket_public_access_block.state_public_access: Creation complete after 2s [id=chvk-tf-state-2026-1028]
aws_s3_bucket_versioning.state_versioning: Creation complete after 3s [id=chvk-tf-state-2026-1028]

Apply complete! Resources: 4 added, 0 changed, 0 destroyed.

Outputs:

lock_table_name = "terraform-state-locks"
state_bucket_name = "chvk-tf-state-2026-1028"
```

Save the actual bucket name. You will use it when configuring the EC2 project's backend.



Task 3 is complete. The S3 bucket has been created by Terraform and is ready to store the EC2 project's state.

7. Task 4 — Configure DynamoDB state locking for Task 2

What is state locking?

Imagine two people run `terraform apply` on the same project at the same time. Both operations might try to update the same state file, which can cause conflicts.

DynamoDB locking coordinates Terraform operations so that only one operation can hold the lock for that state at a time.

- S3: Stores the Terraform state.

- DynamoDB: Coordinates the state lock.
- Terraform: Acquires and releases the lock during supported state-changing operations.

Step 1: Open your original EC2 project

In PowerShell, run:

```
cd C:\IAM policy
code .
```

Open `main.tf`.

Step 2: Add the S3 backend configuration

At the top of the existing `main.tf`, update the `terraform` block to include the backend configuration below. Keep the existing `required_providers` block inside it.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }

  backend "s3" {
    bucket         = "chvk-tf-state-2026-1028"
    key            = "ec2-httpd/"
    region         = "us-east-1"
    encrypt        = true
    dynamodb_table = "terraform-state-locks"
  }
}
```

Important: Replace `YOUR-UNIQUE-BUCKET-NAME` with the exact bucket name created in Task 3. Do not create a second `terraform` block; update the existing one.



Step 3: Understand the backend settings

Setting	Purpose
`bucket`	S3 bucket that stores the state file.
`key`	Path and filename of the state inside the bucket.
`region`	AWS region where the bucket exists.
`encrypt`	Requests encryption for the state object in S3.
`dynamodb_table`	DynamoDB table used for state locking.

Step 4: Initialize the remote backend

Run:

terraform init -migrate-state

What does this command do?

- Detects that the project is changing from local state to S3.
- Copies the existing Terraform state to the configured S3 location when local state is available.
- Configures Terraform to use the remote backend and DynamoDB locking.

If Terraform asks whether you want to copy the existing state, confirm the migration. Do not delete your local state backup until you have verified the remote state.

Expected result: Terraform initializes the S3 backend successfully.

Step 5: Verify the remote state

Run:

terraform state list

```
PS C:\terraform-bootstrap> terraform init -migrate-state
Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v5.100.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

This displays the resources recorded in the state, such as the EC2 instance and security group.

```
PS C:\terraform-bootstrap> terraform state list
aws_dynamodb_table.terraform_locks
aws_s3_bucket.terraform_state
aws_s3_bucket_public_access_block.state_public_access
aws_s3_bucket_server_side_encryption_configuration.state_encryption
aws_s3_bucket_versioning.state_versioning
```

Next, run:

terraform plan

If the configuration and state match the existing infrastructure, Terraform should report that no changes are required.

```
PS C:\terraform-bootstrap> terraform state list
aws_dynamodb_table.terraform_locks
aws_s3_bucket.terraform_state
aws_s3_bucket_public_access_block.state_public_access
aws_s3_bucket_server_side_encryption_configuration.state_encryption
aws_s3_bucket_versioning.state_versioning
PS C:\terraform-bootstrap> terraform plan
aws_s3_bucket.terraform_state: Refreshing state... [id=chvk-tf-state-2026-1028]
aws_dynamodb_table.terraform_locks: Refreshing state... [id=terraform-state-locks]
aws_s3_bucket_public_access_block.state_public_access: Refreshing state... [id=chvk-tf-state-2026-1028]
aws_s3_bucket_versioning.state_versioning: Refreshing state... [id=chvk-tf-state-2026-1028]
aws_s3_bucket_server_side_encryption_configuration.state_encryption: Refreshing state... [id=chvk-tf-state-2026-1028]

No changes. Your infrastructure matches the configuration.

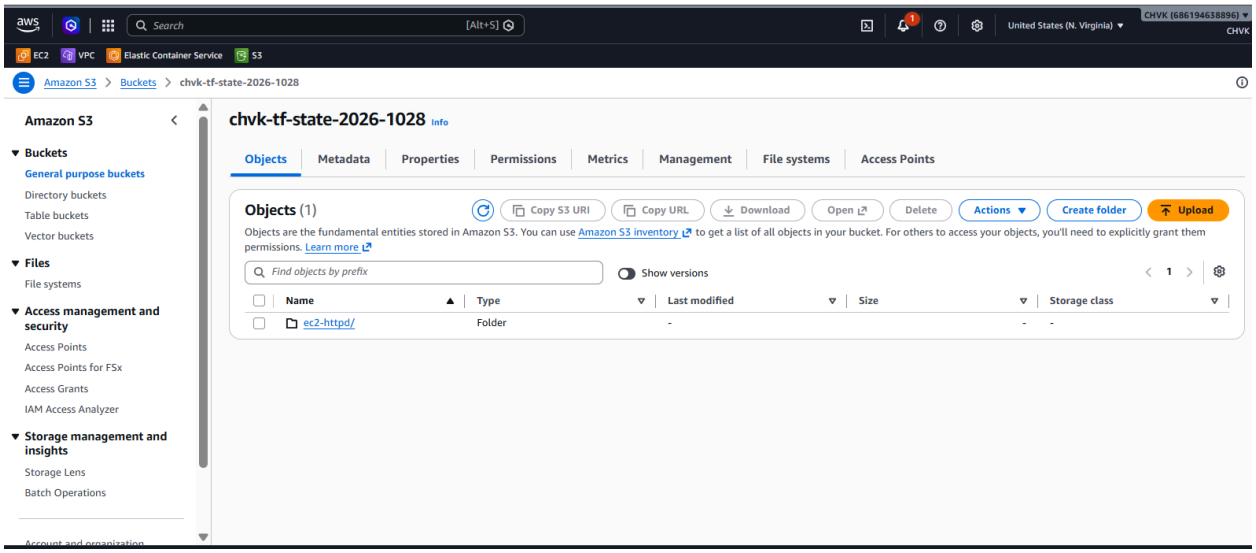
Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.
PS C:\terraform-bootstrap>
```

Step 6: Verify in AWS

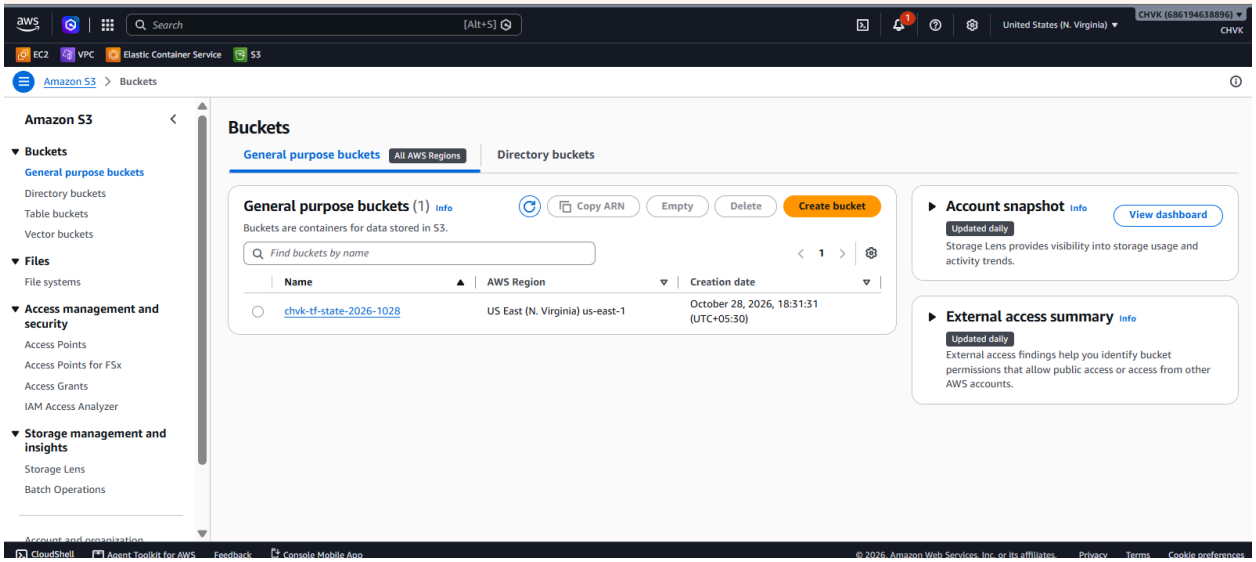
Open the AWS Console and check the following:

Amazon S3

Open your bucket and navigate to ec2-httpd/

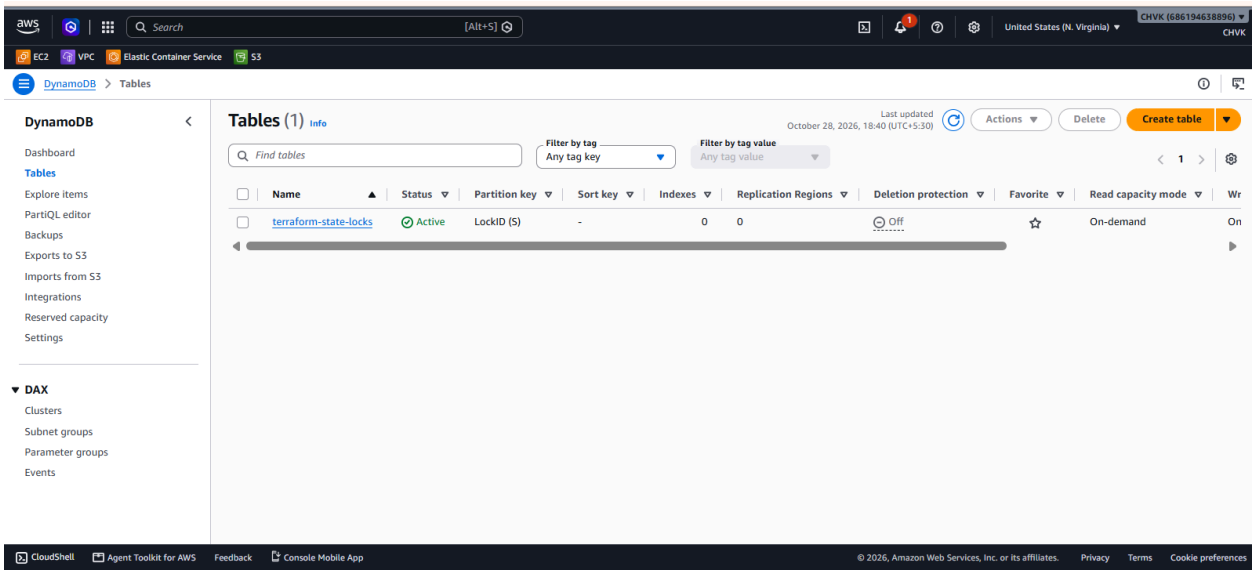


You should see the remote state object after migration.



Amazon DynamoDB

Open the `terraform-state-locks` table.



The table should exist with a partition key named `LockID` of type String. Lock records are temporary and may not be visible when no Terraform operation is running.

Task 4 is complete. Your EC2 project now uses S3 for remote state and DynamoDB for state locking.

8. Complete command reference

Use this table while performing the practical.

Command	Purpose
`terraform init`	Initialize Terraform and download providers.
`terraform validate`	Check configuration syntax.
`terraform plan`	Preview proposed changes.
`terraform apply`	Create or update AWS resources.
`terraform init -migrate-state`	Migrate existing local state to the remote backend.
`terraform state list`	List resources recorded in the current state.

9. Important notes

1. AWS permissions: The identity used by Terraform needs permission to create and manage EC2, security groups, S3, DynamoDB, and the required backend operations.

2. Locking compatibility: The requested `dynamodb_table` option uses the legacy DynamoDB locking mechanism, which is deprecated in newer Terraform versions. For a new setup, current Terraform supports S3-native locking using `use_lockfile = true`. If you choose native locking, enable S3 bucket versioning and remove `dynamodb_table` from the backend configuration; a DynamoDB table is no longer needed for locking.

3. State security: Terraform state may contain sensitive information. Keep the bucket private, use least-privilege IAM permissions, and do not upload state files to public GitHub repositories.

4. Resource cleanup: These resources may incur AWS charges. Do not delete the S3 bucket, DynamoDB table, or state file while the project still depends on them. If you finish the practical and want to remove the resources, first plan the cleanup carefully and preserve anything you need.

10. Final result

After completing all three tasks:

- An EC2 instance runs Amazon Linux 2023 with Apache HTTP Server installed by Terraform.
- An S3 bucket stores the EC2 project's Terraform state remotely.
- A DynamoDB table coordinates Terraform state locking.
- The S3 bucket and DynamoDB table were created automatically using Terraform, not manually through the AWS Console.